从 π_0 到 $\pi_{0.7}$

视觉语言动作模型的技术演进

$\pi_0 \rightarrow \pi_{0.5} \rightarrow \pi^*_{0.6} \rightarrow \pi_{0.7}$

作者：Yang kunpeng

目录

第一章 π_0：连续动作的 Flow Matching VLA **1**
观测、网络结构与动作表示 1
Flow Matching 的监督目标与推理积分 2
预训练、后训练与参数关系 4

第二章 $\pi_{0.5}$：离散预训练与连续动作后训练 **6**
联合概率分解与高低层策略 6
FAST、双重动作监督与参数隔离 7
高层推理、低层生成与执行 9

第三章 $\pi^*_{0.6}$：从机器人经验中进行优势条件学习 **10**
分布式价值函数：从轨迹反馈到连续价值 12
优势指标与条件策略目标 13
RECAP 的训练循环与推理 15

第四章 $\pi_{0.7}$：可操控的多模态条件通用策略 **17**
多模态条件、历史观测与网络结构 17
训练目标：用上下文解释好与坏的动作 19
推理：子任务、目标图像、动作块与实时控制 24
四代技术之间的计算关系 26

第 1 章

π_0：连续动作的 Flow Matching VLA

机器人控制要求策略根据当前观察产生可以执行的连续动作，而预训练视觉语言模型主要处理图像与文字。π_0 将这两个过程连接起来：预训练 VLM 负责理解多模态任务条件，附加的动作专家（action expert）负责生成连续的未来动作序列。该方案同时保留互联网图文预训练带来的语义能力和高频操作需要的连续控制精度。

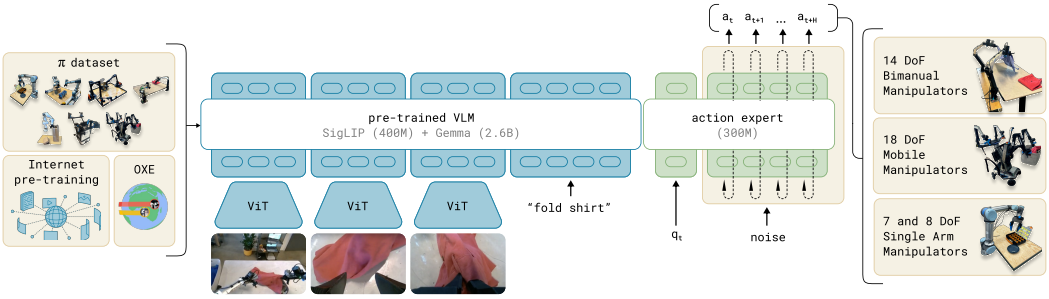


图 1.1: π_0 的总体结构：预训练的 PaliGemma VLM 处理图像与语言；动作专家处理机器人状态和连续动作，并通过 Flow Matching 预测动作块。

观测、网络结构与动作表示

设机器人在控制时刻 t 的观察为

$$\mathbf{o}_t = (I_t^1, \ldots, I_t^n, \ell_t, \mathbf{q}_t), \qquad \mathbf{A}_t = [\mathbf{a}_t, \ldots, \mathbf{a}_{t+H-1}]. \tag{1.1}$$

其中 I_t^i 表示第 i 台相机采集的 RGB 图像，ℓ_t 为语言指令，$\mathbf{q}_t$ 为本体状态（例如关节角度）；$\mathbf{a}_t \in \mathbb{R}^d$ 为一个控制时间步的连续动作，H 是动作块长度。π_0 的主要实验取 $H = 50$。所学习的对象是条件动作块分布 $p_\theta(\mathbf{A}_t \mid \mathbf{o}_t)$，不是把未来动作写成自然语言，也不是逐个回归单个关节位置。

图像经过视觉编码器得到 Patch 特征，语言经过词表嵌入得到文本特征，机器人状态经过可学习的线性投影进入 Transformer 的表示空间。π_0 使用约 3B 参数的 PaliGemma VLM 初始化模型，并添加约 300M 参数的动作专家；动作专家从随机参数

开始训练。两个专家并非两个完全独立、互不通信的网络：它们在对应 Transformer 层中采用不同的计算权重，通过共享的注意力机制交换信息。VLM 处理图文内容；动作专家处理机器人状态以及加噪动作位置。

其注意力结构按三类位置分块：图像与语言位置、机器人状态位置、加噪动作位置。每个块内部允许双向关注，靠前的块不能看到后面的块，动作块则可以利用前面的观测和同块其他动作位置。这个组织方式使得观测前缀与动作去噪过程解耦：在多次 Flow Matching 积分时，图文与状态部分的键值缓存可以复用，重复计算集中在动作专家。

动作专家的输入并不是动作 Token ID。设 $\mathbf{a}^\tau$ 是连续动作在 Flow Matching 时刻 τ 的加噪版本。带时间编码的 MLP 将其投影到动作专家隐藏空间：

$$h^a = W_3 \ \text{swish}(W_2 \, [W_1 \mathbf{a}^\tau ; \phi(\tau)]) \, , \tag{1.2}$$

其中 $\phi(\tau)$ 是正弦时间嵌入，方括号表示特征拼接，W_1, W_2, W_3 是可学习参数。动作专家输出每个动作位置的隐藏向量，再经线性层转换为与整个动作块相同形状的速度场预测，而非直接将输出当作最终执行命令。

Flow Matching 的监督目标与推理积分

给定真实动作块 $\mathbf{A} \sim p_{\text{data}}(\mathbf{A} \mid \mathbf{o})$，采样高斯噪声 $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ 和 Flow Matching 时间 $\tau \in [0, 1]$。采用从噪声到真实动作的线性概率路径：

$$\mathbf{A}^\tau = (1 - \tau)\boldsymbol{\epsilon} + \tau \mathbf{A}. \tag{1.3}$$

对 τ 求导，得到沿这条样本路径的目标速度

$$\frac{\partial \mathbf{A}^\tau}{\partial \tau} = \mathbf{A} - \boldsymbol{\epsilon}. \tag{1.4}$$

因此，训练动作专家 v_θ 的条件 Flow Matching 损失为

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{(\mathbf{o}, \mathbf{A}), \, \tau, \, \boldsymbol{\epsilon}} \left\| v_\theta(\mathbf{A}^\tau, \mathbf{o}, \tau) - (\mathbf{A} - \boldsymbol{\epsilon}) \right\|_2^2 . \tag{1.5}$$

期望对示范数据、Flow Matching 时间及采样噪声共同取值。模型预测的是向量场，不是高斯分布的均值，也不是数据样本的条件概率密度。真实动作参与构造加噪输入与目标速度；训练不需要显式计算动作分布的归一化常数或对数似然。

τ 是生成过程的时间，与机器人控制时刻 t 不同。训练时，τ 从偏向低值（高噪声）的 Beta 分布采样，而非均匀采样。采样分布决定不同噪声区间的训练权重，本身不参与参数学习。

推理时没有专家动作 $\mathbf{A}$。系统先从 $\mathbf{A}^0 \sim \mathcal{N}(0, I)$ 采样一个完整的随机动作块，再沿预测速度场积分：

$$\mathbf{A}^{\tau+\delta} = \mathbf{A}^{\tau} + \delta\, v_\theta(\mathbf{A}^{\tau}, \mathbf{o}_t, \tau). \tag{1.6}$$

使用前向 Euler 积分时，可设 $\delta = 0.1$，经过 10 次迭代从 $\tau = 0$ 积分到 $\tau = 1$。最终 $\mathbf{A}^1$ 才是可进一步反归一化并交给机器人控制器的预测动作序列。动作专家中的 50 个动作位置可以在同一去噪步并行计算，但 Flow Matching 的 10 次积分仍然顺序执行；这和语言 Token 的逐 Token 自回归解码是不同的串行性。

生成的动作块不一定全部执行。对于 50 Hz 机器人，可执行动作块前 25 步；对于 20 Hz 机器人，可执行前 16 步，再采集新观测并生成下一块。因而系统仍具有分块式闭环反馈，而单块内执行的是已经生成的动作序列。动作表示的维度、控制频率与末端或关节控制语义取决于具体机器人平台，不能将 $\mathbf{A}^1$ 不经适配地直接送给其他机械臂。

Flow Matching 的损失计算与动作生成

Flow Matching 有两种等价的时间参数化：式 (1.3) 从 $\tau = 0$ 的噪声积分到 $\tau = 1$ 的动作，目标速度为 $\mathbf{A} - \boldsymbol{\epsilon}$；openpi 采用 $x_t = t\boldsymbol{\epsilon} + (1-t)\mathbf{A}$，从 $t = 1$ 积分到 $t = 0$，目标速度为 $\boldsymbol{\epsilon} - \mathbf{A}$，积分步长为负。两种参数化描述同一条概率路径，速度符号必须与积分方向一致。

函数 `policy(obs, x, t)` 表示以多模态观测为条件的速度场 v_θ，其输出与动作张量 `x` 形状相同。

Listing 1.1: Flow Matching 损失与十步 Euler 动作块生成

```python
"""Flow Matching: t=1 is noise and t=0 is data."""
import torch


def flow_loss(policy, obs, actions):
    # actions: [batch, horizon, action_dim]
    batch = actions.shape[0]
    t = torch.distributions.Beta(1.5, 1.0).sample((batch,))
    t = (0.999 * t + 0.001).to(actions.device)
    noise = torch.randn_like(actions)
    alpha = t[:, None, None]
    x_t = alpha * noise + (1 - alpha) * actions
    target_v = noise - actions
    predicted_v = policy(obs, x_t, t)
    return (predicted_v - target_v).square().mean()


@torch.no_grad()
def sample_chunk(policy, obs, shape, device, steps=10):
    x = torch.randn(shape, device=device)
    dt = 1.0 / steps
    for k in range(steps, 0, -1):
        t = torch.full((shape[0],), k * dt, device=device)
        x = x - dt * policy(obs, x, t)
    return x
```

输入 `actions` 和输出动作块的形状均为 $[B, H, d]$；`policy(obs, x, t)` 应输出相同形状的速度场。训练时并行预测动作块中各位置的速度场；生成时顺序执行多次速度场求值。

预训练、后训练与参数关系

训练首先混合多个机器人构型、操作任务和开源数据集，利用任务名称和片段级语言标注建立多任务行为先验。随后在较高质量、面向特定下游任务的数据上进行后训练，使策略从广泛但质量不一的动作分布向稳定、熟练的任务行为适配。两阶段使用同一个 Flow Matching VLA 主体；主要区别是数据构成、目标任务分布与适配

目的。动作专家的可学习参数与 VLM 主干一起参与机器人训练，并非把 VLM 永久冻结后单独训练一个小型回归器。

连续动作专家建立了动作块的生成机制。$\pi_{0.5}$ 在此基础上增加 FAST 离散动作表示，使离散序列预训练与连续动作后训练形成互补。

第 2 章

$\pi_{0.5}$：离散预训练与连续动作后训练

$\pi_{0.5}$ 要解决的不只是低层运动控制，而是机器人进入未见过的住宅环境后，如何把“整理厨房”等长时序任务分解为可执行的语义子任务，并将不同机器人数据、网络图文数据和动作数据用于同一模型。它继承 π_0 的连续动作专家，同时引入 FAST 离散动作表示和高层文本预测。不同形式的监督共享视觉语言主干，但在低层动作生成时又分离各自的计算路径。

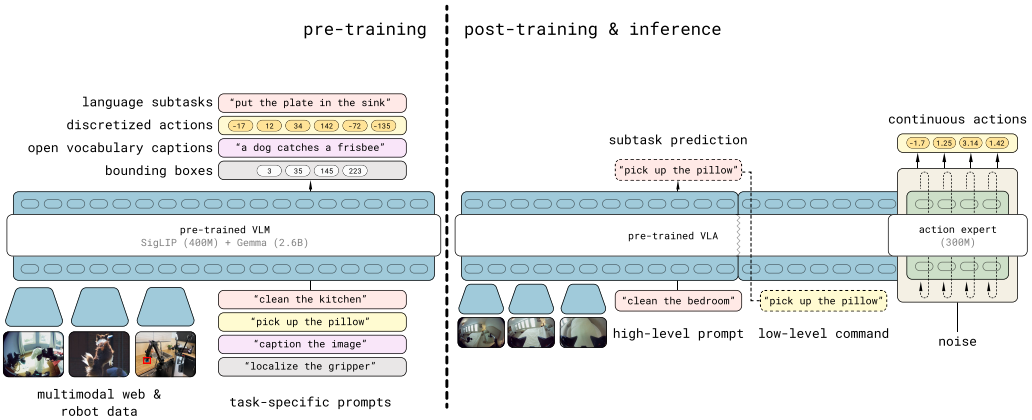


图 2.1: $\pi_{0.5}$ 的两阶段训练：预训练以 FAST 离散动作及其他文本任务进行 Next-Token Prediction；后训练增加 Flow Matching 动作专家，结合高层子任务预测生成低层连续动作。

联合概率分解与高低层策略

模型给定当前观察 $\mathbf{o}_t$ 和总体任务指令 ℓ_t，预测高层子任务文本 $\hat{\ell}_t$ 和未来动作块 $\mathbf{A}_t$。其语义条件分解为

$$\pi_\theta(\mathbf{A}_t, \hat{\ell}_t \mid \mathbf{o}_t, \ell_t) = \pi_\theta(\hat{\ell}_t \mid \mathbf{o}_t, \ell_t)\, \pi_\theta(\mathbf{A}_t \mid \mathbf{o}_t, \hat{\ell}_t). \tag{2.1}$$

这里的 $\hat{\ell}_t$ 是模型产生的具体子任务描述（例如“拿起盘子”），不是低层动作的数值；总体任务 ℓ_t 先影响子任务选择，再通过子任务条件间接影响动作生成。这个分解是一种具体设计约束，而不是任意条件概率链式法则的完整表达，因为它省略了低层策略中对总体任务的直接条件依赖。

模型同时具有文本输出与动作输出。视觉图像经过图像编码器，文本 Token 经过嵌入层；机器人状态在该版本中编码为离散文本 Token。多模态 Transformer 输出用于生成子任务文本和 FAST 动作 Token 的词表 Logits。连续动作专家接收高层条件、图文特征以及当前加噪动作，经线性层输出 Flow Matching 向量场。这样同一策略能够用于低频语义决策和高频动作控制，而不必让高层文字在每个控制时刻重新生成。

FAST、双重动作监督与参数隔离

FAST 是将连续动作块压缩为离散 Token 序列的动作 Tokenizer。其角色不同于 π_0 动作专家输入的连续加噪动作：FAST Token 属于语言模型的离散序列建模，适合大规模数据上的 Next-Token Prediction；连续动作专家则更适合低延迟、高精度的控制。$\pi_{0.5}$ 的训练分为两阶段。

预训练阶段。所有机器人动作统一采用离散 FAST 表示，与图文问答、对象位置、高层子任务等数据共同训练离散 Token 预测模型。机器人状态也作为文本化条件进入序列。这一阶段不优化连续 Flow Matching 项，视觉语义与离散动作序列共同建立预训练表示。

后训练阶段。以已预训练的主干为起点，新增随机初始化的连续动作专家，联合训练文本/FAST 监督与连续 Flow Matching 监督。若将全部需要离散预测的 Token 记为 z，真实连续动作块记为 $\mathbf{A}$，可写成

$$\mathcal{L}_{0.5} = \mathbb{E}\Big[\mathcal{L}_{\mathrm{CE}}(z;\theta) + \alpha \left\| f_\theta^a(\mathbf{A}^\tau, \mathbf{o}_t, \hat{\ell}_t, \tau) - u^\tau \right\|_2^2\Big]. \tag{2.2}$$

其中 $\mathcal{L}_{\mathrm{CE}}$ 是离散词表位置上的交叉熵，α 控制连续动作项相对权重，u^τ 是与所采用的时间方向一致的目标速度；α 是人为指定的超参数。文中后训练设 $\alpha = 10$。在 π_0 的噪声到数据插值 $\mathbf{A}^\tau = (1-\tau)\boldsymbol{\epsilon} + \tau\mathbf{A}$ 下，速度为 $\mathbf{A} - \boldsymbol{\epsilon}$；$\pi_{0.5}$ 的联合损失则把专家输出与 $\boldsymbol{\epsilon} - \mathbf{A}$ 对齐。两种时间方向互为反向，速度场符号与积分方向需要配套定义。

双重监督并不等于让 FAST 生成的动作直接喂给连续专家作为正确答案。模型通过 Attention Mask 分隔 FAST 动作序列与连续动作位置：图像、任务文字及本体

状态组成可共同读取的前缀；FAST 位置按因果顺序关注前序文本和 FAST Token；连续动作位置关注前缀且彼此双向关注，但两种动作表示互相不读取，以避免训练标签泄漏到连续动作预测。连续专家只通过前缀与主干交换信息。

离散与连续监督的损失计算

联合损失中的 `token_logits` 用于子任务文字和 FAST 等离散 Token 的交叉熵，`flow_v` 对应连续动作位置的速度场回归。式 (2.2) 的连续项采用反向时间约定 $u = \boldsymbol{\epsilon} - \mathbf{A}$；`-100` 为不参与交叉熵的掩码标签。

Listing 2.1: $\pi_{0.5}$ 双重监督的联合损失计算

```python
"""Discrete cross-entropy and continuous Flow Matching losses."""
import torch.nn.functional as F


def joint_loss(token_logits, token_targets,
               flow_v, actions, noise, alpha=10.0):
    # token_logits: [B, L, V], targets: [B, L]
    # target -100 is excluded from the token CE
    token_ce = F.cross_entropy(
        token_logits.transpose(1, 2),
        token_targets,
        ignore_index=-100,
    )
    # openpi convention: time 1=noise, time 0=data
    target_v = noise - actions
    continuous_loss = (flow_v - target_v).square().mean()
    return token_ce + alpha * continuous_loss
```

离散交叉熵和连续 Flow Matching 损失分别作用于对应的输出位置。FAST Token 序列与动作专家之间的可见性由 Attention Mask 控制；损失相加并不改变两条分支的注意力约束。

高层推理、低层生成与执行

推理分为两个不同频率的循环。高层根据 $\mathbf{o}_t$ 与 ℓ_t 自回归预测文本子任务 $\hat{\ell}_t$，低层在该子任务条件下使用动作专家预测连续动作块。低层从高斯噪声动作块出发，经过多步 Flow Matching 积分取得连续控制命令；其中的 FAST 离散动作 Token 主要用于训练，不是低层实时执行时必须先解码的一条串行路径。低层推理采用 10 次去噪/积分步骤。

模型在训练数据中同时看到关节空间和末端空间的控制目标，使用控制模式文本标识区分表示。动作各维度按所属数据集统计的第 1 与第 99 百分位数归一化到指定数值范围；不足最大动作维度的机器人使用零填充。部署必须依据所选机器人与控制模式执行反归一化和控制接口转换。这一高低层分解为下一阶段使用价值函数评估“哪类动作更有利于完成任务”提供了可扩展基础。

第 3 章

$\pi^*_{0.6}$：从机器人经验中进行优势条件学习

单纯模仿专家数据无法充分利用机器人自主执行时的失败、修正和速度差异。$\pi^*_{0.6}$ 将学习问题扩展到真实机器人经验：先用价值函数估计行动前后向任务完成推进了多少，再将“动作是否更优”表示为文本条件输入 VLA，通过优势条件训练提取更优策略。对应算法称为 RECAP（RL with Experience and Corrections via Advantage-conditioned Policies）。这不是在推理时重新规划一个数值奖励最大化轨迹，而是在训练时利用奖励评价改写动作的条件分布。

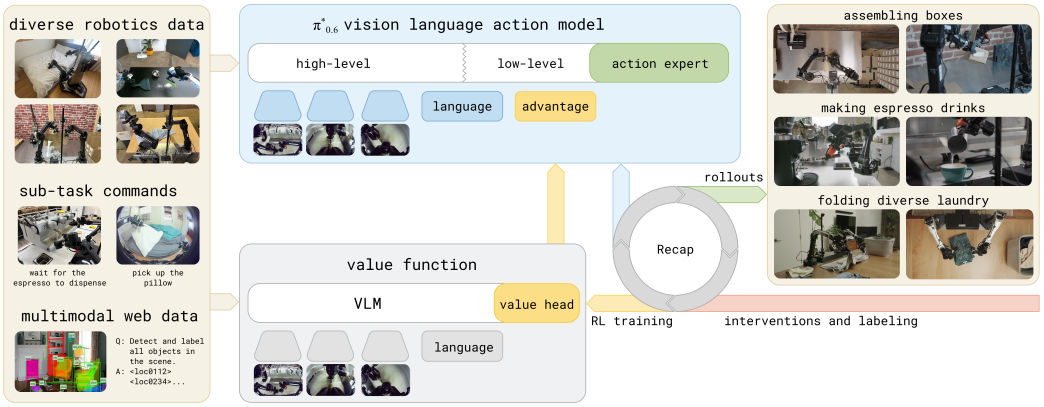


图 3.1: $\pi^*_{0.6}$ 的训练流程：多源机器人数据、价值函数、优势条件 VLA 与 RECAP 迭代训练。

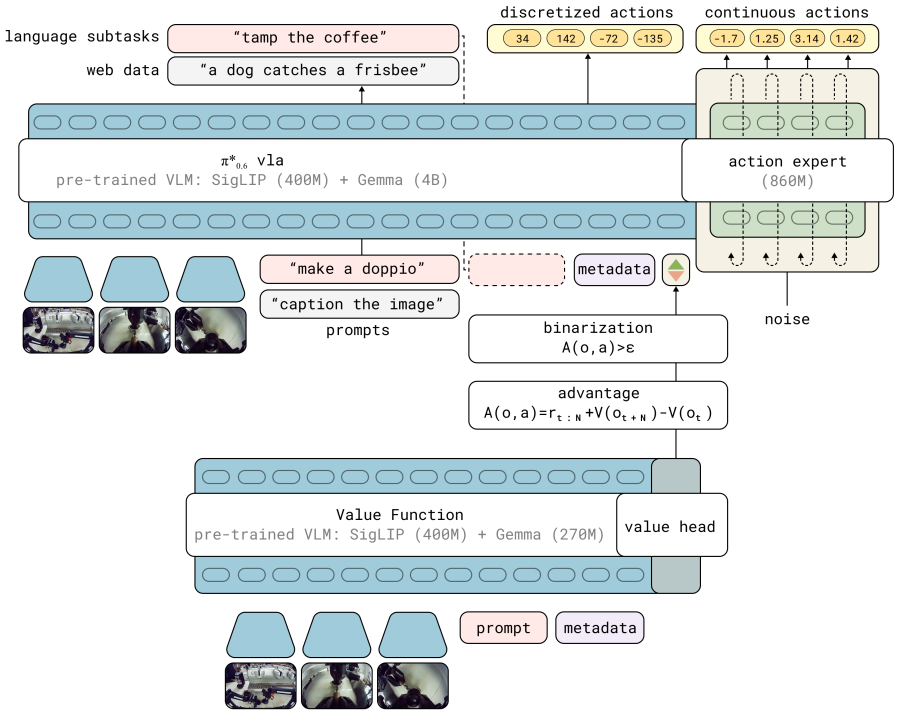


图 3.2: $\pi^*_{0.6}$ 的 VLA 与价值函数：价值网络将经验轨迹中的回报映射为价值分布，计算优势指标并送入策略前缀；策略保持文本/FAST 与连续动作专家两种输出路径。

分布式价值函数：从轨迹反馈到连续价值

采集数据由专家示范、自主执行及人工接管纠正构成。对于轨迹 $\xi = (\mathbf{o}_0, \mathbf{a}_0, \ldots, \mathbf{o}_T)$，奖励 r_t 由任务结果和执行进度定义。奖励采用稀疏终点反馈与时间惩罚：

$$r_t = \begin{cases} 0, & t = T \text{ 且成功}, \\ -C_{\text{fail}}, & t = T \text{ 且失败}, \\ -1, & \text{其他时刻}. \end{cases} \tag{3.1}$$

因此成功轨迹越快完成，剩余累积回报越接近零；失败轨迹额外遭受较大惩罚。由于不同任务时长有差异，回报按任务最大长度归一化，使价值尺度大致处于 $(-1, 0)$。

使用行为数据上的经验回报

$$R_t(\xi) = \sum_{k=t}^{T} r_k \tag{3.2}$$

作为价值监督，先将其离散化到 $B = 201$ 个回报区间。价值网络输出 $p_\phi(b \mid \mathbf{o}_t, \ell)$ $(b = 1, \ldots, B)$，每一项表示回报落入对应区间的预测概率。该网络不是直接回归一个实数，而是预测价值分布；其训练损失为

$$\mathcal{L}_V(\phi) = \mathbb{E}_{\xi \sim \mathcal{D}} \sum_{t \in \xi} -\log p_\phi\big(b(R_t(\xi)) \mid \mathbf{o}_t, \ell\big). \tag{3.3}$$

其中 $b(\cdot)$ 把真实回报映射到离散区间；期望对已有轨迹采样求取，求和覆盖轨迹内有效时刻。预测完成后，可将离散分布转换为连续标量估计：

$$V_\phi(\mathbf{o}_t, \ell) = \sum_{b=1}^{B} p_\phi(b \mid \mathbf{o}_t, \ell)\, v_b. \tag{3.4}$$

v_b 是第 b 个回报区间的代表值。$\pi^*_{0.6}$ 使用较小的 VLM 初始化价值模型，其参数 ϕ 与动作策略 θ 分开训练；价值模型提供优势标签，不需要在每次推理机器人动作时在线调用。

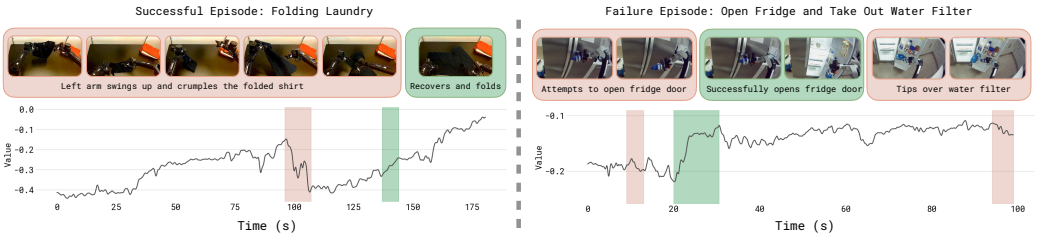


图 3.3: 价值函数可视化：价值变化反映任务进展、错误与完成速度。

优势指标与条件策略目标

利用价值函数可以估计某条训练动作相对于当前状态平均未来回报的优势。以 N 步近似为例，

$$\hat{A}_t = \sum_{j=t}^{t+N-1} r_j + V_\phi(\mathbf{o}_{t+N}, \ell) - V_\phi(\mathbf{o}_t, \ell). \tag{3.5}$$

$\hat{A}_t$ 越大，说明该动作片段带来的回报或进度越高。根据任务相关阈值 ϵ_ℓ，将连续优势转为布尔指标

$$I_t = \mathbb{I}\{\hat{A}_t > \epsilon_\ell\}. \tag{3.6}$$

预训练阶段按任务设置阈值，使示范数据中约 30% 的样本具有正优势；任务专项微调时，通常将阈值设为每轮评估轨迹中约 40% 的样本为正优势。对 T-shirt 与短裤折叠任务，正优势样本比例约为 10%，以强调高质量、较快的行为。后训练优势估计使用 $N = 50$ 步前瞻；预训练时使用整条 Episode 的回报估计，即 $N = T$。人工纠正动作被直接标为正优势，以学习专家恢复操作。

这个指标被写成普通的输入文本条件（`Advantage: positive` 或 `Advantage: negative`），插在高层子任务描述之后、动作预测位置之前。如此处理可以复用 VLM 条件序列结构，而不需要另建一个连续数值 Advantage 输入网络。理论上的优势条件策略可写为

$$\hat{\pi}(\mathbf{a} \mid \mathbf{o}, \ell) \propto \pi_{\text{ref}}(\mathbf{a} \mid \mathbf{o}, \ell) \left[\frac{\pi_{\text{ref}}(\mathbf{a} \mid I, \mathbf{o}, \ell)}{\pi_{\text{ref}}(\mathbf{a} \mid \mathbf{o}, \ell)} \right]^\beta, \tag{3.7}$$

π_{ref} 是示范与历史自主策略组成的行为参考分布，β 控制偏向正优势动作的强度。$\beta = 1$ 时，目标退化为正指标条件下的动作分布。训练中可以随机去掉指标，让模

型学习有指标和无指标的条件分布；这种设计也为推理时的 Classifier-Free Guidance（CFG）提供条件。

优势条件策略的训练目标写为

$$\begin{aligned}\min_{\theta} \quad & \mathbb{E}_{\mathcal{D}_{\pi_{\text{ref}}}}\left[-\log \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_t, \ell) - \alpha \log \pi_\theta(\mathbf{a}_t \mid I_t, \mathbf{o}_t, \ell)\right], \\ & I_t = \mathbb{I}\left(A^{\pi_{\text{ref}}}(\mathbf{o}_t, \mathbf{a}_t, \ell) > \epsilon_\ell\right).\end{aligned} \tag{3.8}$$

第一个项拟合无优势条件的行为分布，第二个项拟合带优势条件的行为分布。训练保留正、负优势数据，并通过条件标签区分行为，而不是直接删除失败轨迹。实际 VLA 的离散输出使用 Token 交叉熵，连续输出使用 Flow Matching 目标；上式是策略提取目标的似然表达，不表示代码中直接计算连续动作的精确对数似然。

训练时以 30% 的概率丢弃优势条件，使同一策略既能生成带条件的动作，也能生成无条件的动作。初始目标任务的示范 SFT 将所有示范的 I_t 固定为正值。推理时可直接使用正优势条件；也可将有条件和无条件预测组合进行 Classifier-Free Guidance（CFG），以增强对正优势行为的偏好。过大的 CFG 权重可能把动作推向训练分布支持范围的边缘，行为选择主要由优势阈值控制，并可在需要时使用适度 CFG。

式 (3.7) 描述理想分布下的策略提取目标。在有限数据、有限模型容量和近似价值函数下，实际策略质量仍取决于优势标签的准确性与优化过程。

价值分布与优势标签的计算

价值网络先产生 B 个回报区间的 Logits。对其做 Softmax，再以区间代表值加权求和，即得到式中使用的 V_ϕ。一个训练片段的 N 步优势由奖励和前后两个状态的估值计算，最后用任务阈值生成正负文本条件。

Listing 3.1: 价值期望、N 步优势及纠正动作标签（公式计算示例）

```python
"""Value expectation, N-step advantage, and correction labels."""
import torch


def expected_value(value_logits, bin_centers):
    # value_logits: [..., num_bins]
    probs = torch.softmax(value_logits, dim=-1)
    return (probs * bin_centers).sum(dim=-1)

```

```
10
11 def advantage_labels(rewards, values, horizon,
12                       threshold, is_correction):
13     # rewards: [T]; values: [T+1]
14     # is_correction: [T] boolean indicators
15     assert len(values) == len(rewards) + 1
16     estimates = []
17     for t in range(len(rewards)):
18         end = min(t + horizon, len(rewards))
19         advantage = (rewards[t:end].sum()
20                      + values[end] - values[t])
21         estimates.append(advantage)
22     estimates = torch.stack(estimates)
23     positive = (estimates > threshold) | is_correction
24     return estimates, positive
```

阈值 `threshold` 由每个任务的优势分布标定，不随当前 Batch 动态变化。`value[t]` 与 `value[end]` 对应同一任务的状态价值；人工纠正标记只修改最终优势类别，不改变数值优势。

RECAP 的训练循环与推理

Algorithm 1 RL with Experience and Corrections via Advantage-conditioned Policies (RECAP)

Require: multi-task demonstration dataset $\mathcal{D}_{\text{demo}}$

Train V_{pre} on $\mathcal{D}_{\text{demo}}$ using Eq. (3.3)
Train π_{pre} on $\mathcal{D}_{\text{demo}}$ using Eq. (3.8) and V_{pre}
Initialize $\mathcal{D}_\ell$ with demonstrations for ℓ
Train V_ℓ^0 from V_{pre} on $\mathcal{D}_\ell$ using Eq. (3.3)
Train π_ℓ^0 from π_{pre} on $\mathcal{D}_\ell$ using Eq. (3.8) and V_ℓ^0
for $k = 1$ to K **do**
    Collect data with π_ℓ^{k-1}, add it to $\mathcal{D}_\ell$
    Train V_ℓ^k from V_{pre} on $\mathcal{D}_\ell$ using Eq. (3.3)
    Train π_ℓ^k from π_{pre} on $\mathcal{D}_\ell$ using Eq. (3.8) and V_ℓ^k
end for

RECAP 在每一轮任务专项训练中从预训练检查点初始化价值模型与策略，再使用累计的数据进行训练，以控制多轮迭代的分布漂移。流程由示范初始化、机器

人自主采样、人工接管纠正、累计轨迹重新估值和优势条件训练构成。负优势样本保留在数据中，用于区分不同优势条件下的行为分布。

$\pi^*_{0.6}$ 基于 $\pi_{0.6}$ 而非直接复用 $\pi_{0.5}$ 的所有内部参数。它保留高层子任务文本预测，以及 FAST 离散动作与 Flow Matching 连续动作专家的双路径结构，同时增加优势条件。训练采用 KI（Knowledge Insulation）原则：动作专家从 VLM 激活中获取条件，但其连续损失的梯度不会反向更新 VLM 主干；主干主要通过离散 Token 交叉熵学习。因此 KI 与 π_0 的端到端联合训练采用不同的梯度传播路径。

推理时模型根据任务条件、当前观测和所选优势指标预测子任务及连续动作块。价值网络主要用于离线估值、构造训练目标；常规动作推理不依赖每一步现场重新估值。对于正优势条件，可以直接用条件模型采样；在需要时还可以将有条件与无条件速度场组合做 CFG，但这会增加相应推理计算负担。

训练与推理的输入输出

模块	训练输入与监督	推理输入与输出
价值模型	观测、任务、轨迹累计回报；输出 201-bin 价值分布并计算交叉熵。	通常不进入实时动作路径；不需要每步调用。
VLM 主干	图像、状态、任务/子任务、优势文本及 FAST/文本 Token；通过离散 Token 交叉熵训练。	实时观测与任务条件；输出子任务文本及 VLM 条件特征。
Action Expert	带噪真实动作块、Flow 时间和 VLM 特征；预测向量场并计算 Flow Matching 损失。	从随机噪声动作块开始多步去噪，输出连续动作块。
RECAP 循环	示范数据初始化；自主执行与人工纠正数据累积后，重新估值并训练策略。	执行策略、采集轨迹；训练在批次间迭代，不在线更新参数。

第 4 章

$\pi_{0.7}$：可操控的多模态条件通用策略

前一阶段利用成功、失败与人工纠正学习如何趋向更好的行为，$\pi_{0.7}$ 则进一步扩展任务条件的表达能力。单一任务文字很难完整描述复杂技能的执行速度、细节步骤与期望的近未来图像；把这些信息显式加入条件，可以利用质量不一、来源不同的数据，也使部署者能够在不进行专项后训练的情况下调节策略行为。

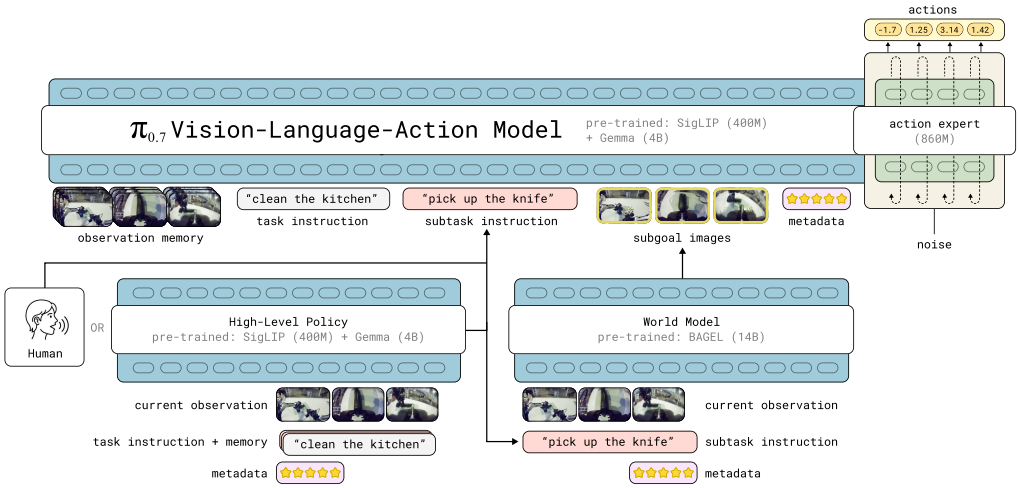


图 4.1: $\pi_{0.7}$ 的架构：视频历史编码器、视觉语言主干、连续动作专家及额外的子任务文字、目标图像和轨迹元信息；运行时可由高层策略和世界模型动态提供条件。

多模态条件、历史观测与网络结构

模型的条件不再只是总体任务说明 ℓ_t。设运行时条件

$$\mathcal{C}_t = (\ell_t, \hat{\ell}_t, \mathbf{g}_t, m_t, c_t), \tag{4.1}$$

分别表示总体任务、当前子任务、目标图像、轨迹元信息及控制模式。$\hat{\ell}_t$ 可以由高层语言策略生成，也可以由人类实时指导；$\mathbf{g}_t$ 是一个或多个相机视角中期望达到的近

未来图像；m_t 包含任务执行时长、质量评分和错误标记等轨迹属性；$c_t \in \{\texttt{joint}, \texttt{ee}\}$ 指示所需的关节或末端空间控制方式。

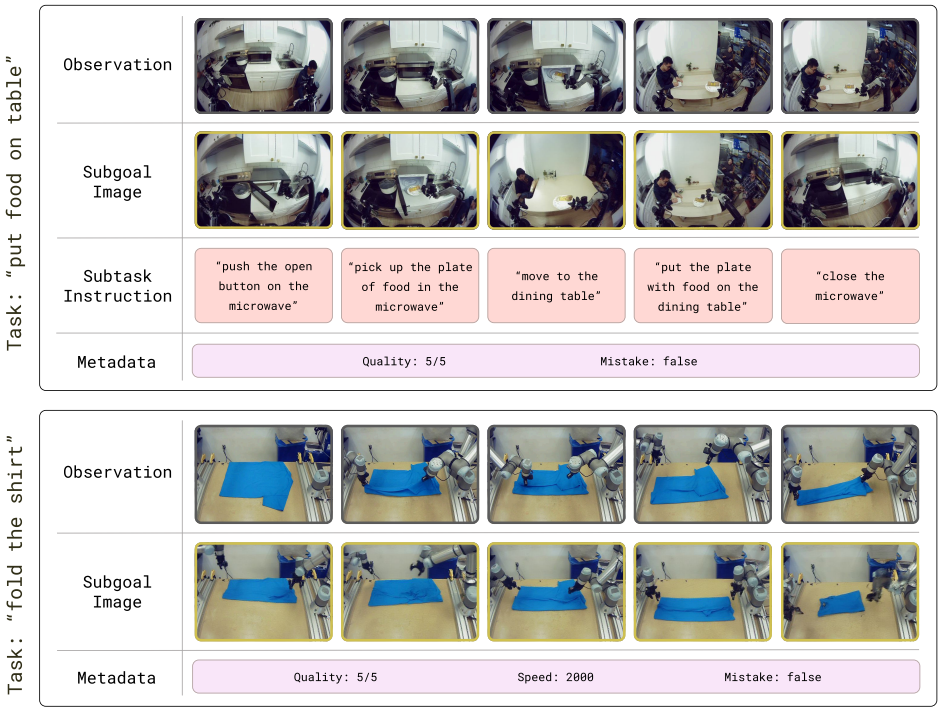


图 4.2: $\pi_{0.7}$ 的条件组成：整体任务、子任务、目标图像、速度/质量/错误属性和控制模式共同指定希望执行的行为。

网络约有 5B 参数，由 Gemma 3 4B VLM 初始化的视觉语言主干、具有时间/空间压缩的 MEM 风格历史编码器，以及约 860M 参数的 Flow Matching 动作专家组成。模型最多接收四路相机图像（前视、两个腕部视角、可选后视），每路最多使用六帧历史图像；历史图像压缩到与单帧相同的 Token 数量。最多还可使用三个视角的目标图像。输入图像经过 448×448 处理；状态历史不再像 $\pi_{0.6}$ 那样离散为文字 Token，而是经过可学习的线性投影，以连续状态 Embedding 形式进入主干。

视觉观察与目标图像分别在块内部进行双向注意力；目标图像位置还能访问当前观察；后续文本部分按因果注意力规则处理。动作专家输入固定 50 个动作位置，动作位置内部双向关注且可访问 VLM 激活。$\pi_{0.7}$ 继续使用 KI：动作专家的 Flow Matching 梯度不反向流入 VLM 主干，主干主要依赖 FAST 和其他离散监督进行学习。

训练与推理的输入输出

项目	训练时	推理时
图像与状态	数据集中的当前/历史相机图像、机器人状态。	相机与机器人实时提供的当前/历史观测。
任务条件	总任务、片段级子任务及轨迹元信息。	用户任务；子任务由高层策略或人类提供；元信息指定期望行为。
目标图像	可选的真实未来图像或 World Model 生成图像。	可选，由 World Model 异步生成；也可以完全不提供。
动作输入	真实动作块经加噪后的表示 $\mathbf{A}^\tau$ 与 Flow 时间。	从高斯噪声动作块开始去噪；RTC 可附加已提交动作前缀。
模型直接输出	VLM 输出离散 Token Logits；Action Expert 输出向量场预测。	经过多步 Flow 去噪后得到长度为 50 的连续动作块。
参数更新/执行	用离散交叉熵和 Flow Matching 损失更新参数。	不更新模型参数；每轮执行动作块中的 15 或 25 步，再读取新观测。

MEM 历史视觉编码器

模型最多接收四路相机图像，每路最多六帧历史图像，按 1 秒间隔采样；图像调整为 448×448。历史编码器采用 MEM 风格的时空压缩：历史帧先在视觉编码器内融合，再输出与单帧相同数量的视觉 Token。因此，增加历史帧不会让送入 VLM 的视觉 Token 数按帧数线性增长。

历史状态同样进入主干，但不是离散文本 Token。每一帧的本体状态经线性投影映射到主干维度，并作为独立 State Token；若对应历史图像被丢弃，该状态 Token 也被屏蔽。训练时以 30% 概率丢弃整段历史；可用的后视相机图像另以 30% 概率丢弃。

训练目标：用上下文解释好与坏的动作

$\pi_{0.7}$ 允许训练数据含成功、失败、修正及不同速度的轨迹，但并不是把所有动作都无差别地当作“正确执行”。任务指令与轨迹元信息为训练行为提供解释：同样的

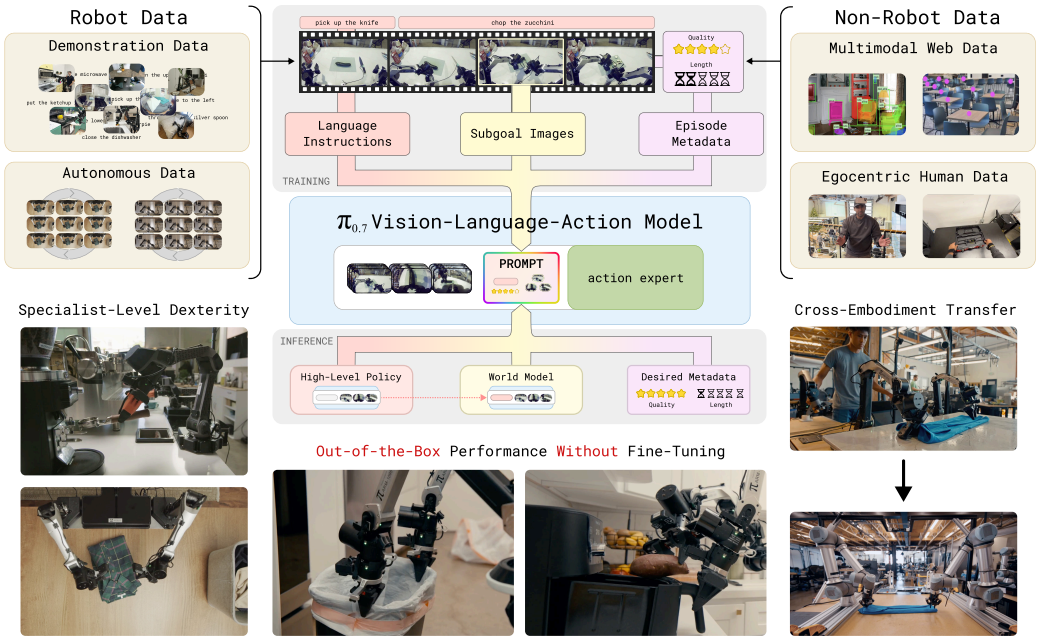


图 4.3: $\pi_{0.7}$ 的训练数据与推理条件：机器人数据、非机器人数据、语言指令、目标图像和 Episode Metadata。

视觉场景与动作，在 `Quality: 5`、`Mistake: false` 与低质量记录下有不同的条件含义。在运行时，模型根据任务条件、当前/历史观测和 Metadata 生成动作块。质量与错误标签是训练数据元信息，不是模型在动作推理时自动求出的真实成功率或额外奖励函数。训练目标可写为条件动作似然：

$$\max_{\theta} \ \mathbb{E}_{\mathcal{D}} \left[\log \pi_\theta(\mathbf{A}_{t:t+H} \mid \mathbf{o}_{t-T:t}, \mathcal{C}_t) \right]. \tag{4.2}$$

连续动作由 Flow Matching 动作专家训练；VLM 主干同时使用 FAST 动作 Token 和其他文本/视觉语言任务上的离散交叉熵训练。

Episode Metadata 与条件丢弃

训练数据包含不同质量、不同速度、成功与失败的轨迹。Metadata 为这些轨迹提供行为语境：

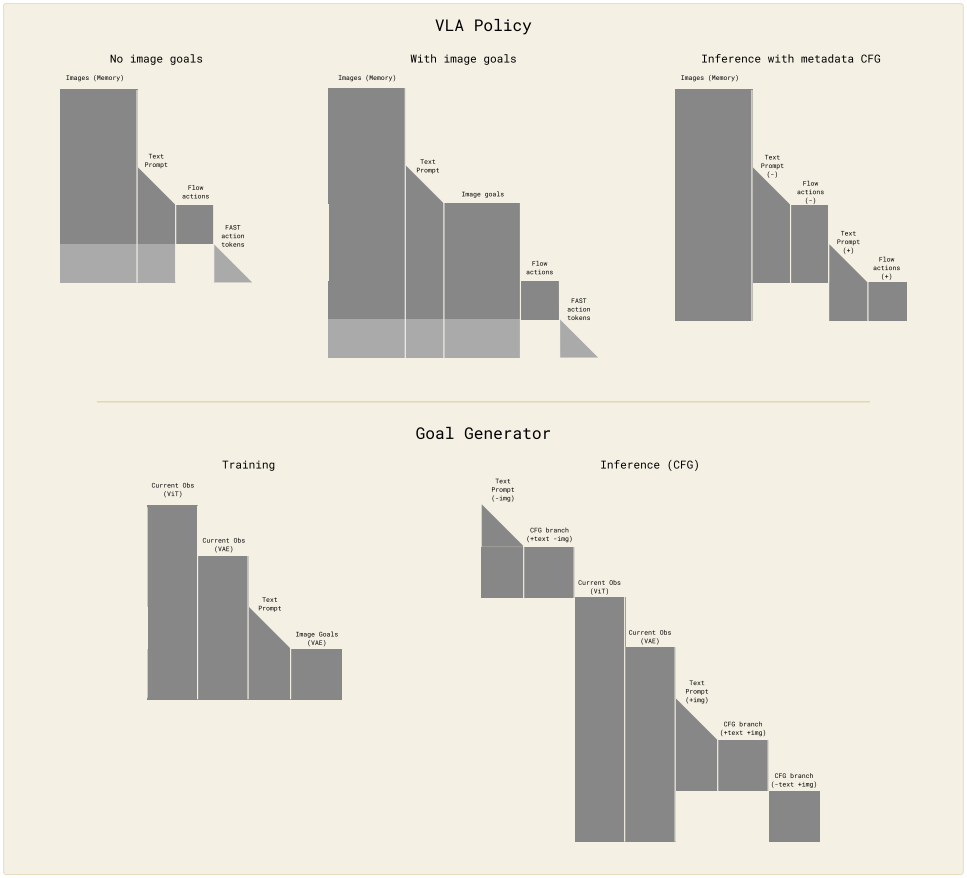


图 4.4: 注意力掩码：历史观测、目标图像、文本、FAST Token、连续动作及 World Model 之间的注意力可见关系。

字段	含义与用法
Overall Speed	Episode 的总步数，按 500 步为间隔离散化；例如 1750–2250 步归入 “2000 steps”。
Overall Quality	人工标注的整体执行质量，取值 1–5，5 为最高质量。
Mistake	人工标注的动作片段是否出现错误，例如抓取失败或执行了错误子任务。
Control Mode	`joint` 或 `ee`，分别指定关节空间或末端执行器空间控制；训练时不对该字段做 Dropout。

训练时，目标图像只加入约 25% 的样本；有目标图像的样本中，子任务文本以 30% 概率丢弃。整组 Metadata 以 15% 概率丢弃，Speed、Quality、Mistake 三个字段各自再以 5% 概率独立丢弃。历史图像以 30% 概率整体丢弃，后视相机图像（若有）另以 30% 概率丢弃。条件丢弃使模型能够在推理时使用完整条件，也能够在缺少目标图像或部分 Metadata 时运行。

部署时，Overall Quality 设为 5、Mistake 设为 false；Overall Speed 设为对应任务 Episode 长度的第 15 百分位数。Control Mode 根据机器人实际控制接口选择。

目标图像是模型希望场景达到的近未来视觉状态。它由单独的 World Model 生成；World Model 不输出机器人底层动作，训练细节如下。

World Model 的训练数据与推理

World Model 与 VLA 分开训练。每个 World Model 训练样本包含子任务文本、当前三个相机视角和对应的三个目标图像；目标图像取自覆盖当前时刻的子任务片段结束时刻：

$$\mathbf{g}_t^* = \mathbf{o}_{t_{\text{end}}}. \tag{4.3}$$

模型从 BAGEL（14B 参数）初始化。当前图像同时经过 ViT 与 VAE：ViT 分支负责语义理解，输入分辨率为 448×336；VAE 分支保留细粒度图像信息，当前图像和目标图像的输入分辨率为 512×384。ViT Token 进入 7B 语言主干，VAE Token 进入 7B 生成主干。训练数据包含高质量片段标注的机器人轨迹、第一人称人类视频、开源图像编辑数据与视频数据；子任务时间边界和文本标签质量直接影响目标图像质量。

推理时，World Model 的输入是当前多视角图像、子任务文本和 Metadata，输出是多视角目标 RGB 图像。生成目标图像时使用 25 个去噪步骤和 CFG。目标图像

在子任务变化时更新，或在距上次生成约 4 秒时更新；生成过程异步运行，不阻塞 VLA 执行。

VLA 训练时，约 25% 的样本提供目标图像。在这些真实目标图像样本中，25% 取子任务结束帧，75% 从当前时刻之后 0–4 秒内均匀抽取未来帧；训练还加入 World Model 生成的目标图像，以缩小真实未来图像与推理生成图像之间的差异。目标图像是可选条件，不是动作监督答案；动作监督仍然来自真实 Action Chunk。

多模态条件的样本组织

$\pi_{0.7}$ 将历史观察、子任务、可选的目标图像、行为偏好和控制模式组成统一的条件结构。其中图像历史与状态描述当前观测，目标图像描述期望的近未来视觉状态，元信息控制质量、错误类型与执行速度。

Listing 4.1: $\pi_{0.7}$ 多模态控制条件的字段组织示意

```
"""Conditional inputs for a multimodal action policy."""


def make_context(task, subtask, image_history,
                 robot_state, control_mode,
                 goal_image=None, quality=5,
                 mistake=False, speed="normal"):
    if control_mode not in ("joint", "ee"):
        raise ValueError("Invalid control mode")
    return {
        "task": task,
        "subtask": subtask,
        "image_history": image_history,
        "robot_state": robot_state,
        "goal_image": goal_image,
        "metadata": {
            "quality": quality,
            "mistake": mistake,
            "speed": speed,
        },
        "control_mode": control_mode,
    }
```

其中 goal_image=None 表示本轮没有目标图像条件；control_mode 必须和动作数值的实际含义对应。图像编码器、历史记忆模块、语言主干和 Flow Matching 动作专家据此形成动作预测。quality 等字段在这里仅表示期望的行为属性，不是当前动作真实成功概率。

推理：子任务、目标图像、动作块与实时控制

推理时，主策略根据近期图像与状态历史、总体任务和动态更新的条件，生成下一段连续动作。子任务可由高层策略产生或人工指导；目标图像由世界模型异步生成，子任务变化或距离上次生成超过约四秒时刷新。元信息通常指定高质量、无错误，并按任务设置速度偏好。动作模型每次使用 5 次 Flow Matching 去噪步骤生成长度为 50 的动作块，再执行其中 15 或 25 个控制步，结合新观察继续预测。

Adaptive RMSNorm 与 Training-Time RTC

Action Expert 使用 Adaptive RMSNorm 注入 Flow Matching 时间信息，使内部特征调制随去噪阶段变化。RTC 训练允许每个 Action Token 使用不同的 Flow 时间，因此模型可以同时接收干净的已知动作前缀与带噪的待生成后缀。

RTC 处理的是动作块异步生成造成的新旧 Chunk 衔接问题。设预测长度为 H，每次执行 s 步，生成新 Chunk 的延迟为 d 步。只有当

$$d \leq H - s \tag{4.4}$$

时，旧 Chunk 才有足够的重叠动作覆盖新 Chunk 的推理延迟。延迟期间已经提交的动作构成 Prefix，其余动作构成需要生成的 Postfix。

Training-Time RTC 在训练时随机模拟延迟，并学习条件分布

$$p_\theta\left(\mathbf{A}_{t+d:t+H} \mid \mathbf{o}_t, \mathbf{A}_{t:t+d}\right). \tag{4.5}$$

训练使用的 Flow 路径为

$$\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1-\tau)\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{4.6}$$

其中 $\tau = 0$ 对应噪声，$\tau = 1$ 对应干净动作。与该路径一致的目标速度为 $\mathbf{A}_t - \boldsymbol{\epsilon}$。训练时将前 d 个 Prefix 动作直接设为真实动作，并把对应 Token 的 Flow 时间设为 1；Postfix 按常规 Flow Matching 加噪。损失只在 Postfix 上计算：

$$\mathcal{L}_{\text{RTC}} = \frac{\sum_{i=d}^{H-1} \left\| \hat{\mathbf{v}}_i - (\mathbf{a}_i - \boldsymbol{\epsilon}_i) \right\|_2^2}{H - d}. \tag{4.7}$$

$\pi_{0.7}$ 在训练中模拟 $d \in \{0, 1, \ldots, 12\}$ 的延迟；在 50 Hz 控制频率下，12 步对应 240 ms。推理时，机器人继续执行已提交的旧动作，Action Expert 以该动作 Prefix 为条件生成 Postfix；新 Chunk 返回后，从尚未执行的部分接续。Training-Time RTC 将动作前缀条件化放在训练阶段，不需要在每个去噪步骤额外执行推理时 Inpainting Guidance。

算法中的 $\mathbf{a}_{t:}$ 表示异步再规划时需要考虑当前尚未执行完的动作队列；RTC（Real-Time Chunking）用于在推理延迟与连续动作块切换之间保持轨迹一致性。训练时模拟 0–12 个控制步的延迟，使模型适应非零推理延迟。生成的 50 步动作块通常只执行其中一段，实际执行情况需在下一次观测中反馈给策略。

Algorithm 2 Prompting $\pi_{0.7}$ at test time

Input: initial observation $\mathbf{o}_0$, task instruction ℓ, episode metadata m, control mode c
Initialize subtask $\hat{\ell}$ (from high-level policy or coaching)
if using subgoal images **then**
    $\mathbf{g}^\star \sim p_\psi(\mathbf{g}^\star \mid \mathbf{o}_0, \hat{\ell}, m)$
    $\mathcal{C} = \{\ell, \hat{\ell}, \mathbf{g}^\star, m, c\}$
else
    $\mathcal{C} = \{\ell, \hat{\ell}, m, c\}$
end if
$\mathbf{a}_{t:t+H} \sim \pi_\theta(\mathbf{a} \mid \mathbf{o}_{t-T:t}, \mathcal{C})$ ▷ Optional: CFG
for $t = 0, 1, 2, \ldots$ **do**
    if using subgoal images and ($\hat{\ell}$ changed **or** Δ-second timer elapsed) **then**
        $\mathbf{g}^\star \sim p_\psi(\mathbf{g}^\star \mid \mathbf{o}_t, \hat{\ell}, m)$ ▷ Non-blocking (async)
        $\mathcal{C} = \{\ell, \hat{\ell}, \mathbf{g}^\star, m, c\}$
    end if
    if $\hat{H}$ steps elapsed since last inference **then**
        $\mathbf{a}_{t:t+H} \sim \pi_\theta(\mathbf{a} \mid \mathbf{o}_{t-T:t}, \mathcal{C}, \mathbf{a}_{t:})$ ▷ Async w/ RTC
    end if
    Execute $\mathbf{a}_t$
end for

由于训练时包含条件 Dropout，推理可用有条件和无条件速度场进行 Classifier-Free Guidance。直观地说，它在不同条件预测之间作加权，使模型更偏向指定的速度、质量或其他控制偏好；权重加大也可能带来过激动作，且额外的条件分支产生计算开销。实际运行采用异步高层和目标图像生成，底层 VLA 可以持续使用最新可用的条件而不必等待世界模型完成一次生成。

四代技术之间的计算关系

四个版本不是简单地逐代增加模型参数，而是在同一类 VLA 框架中逐步解决不同的约束。

版本	主要技术变化
π_0	VLM 提供任务条件；独立 Action Expert 用 Flow Matching 生成连续动作块。
$\pi_{0.5}$	FAST 离散动作预训练与连续动作后训练相衔接；新增高层子任务文本预测。
$\pi^*_{0.6}$	在 $\pi_{0.6}$ 基础上引入价值网络、优势指标和从自主经验/人工纠正中学习的 RECAP 过程。
$\pi_{0.7}$	引入目标图像、历史记忆、轨迹质量/速度/错误元信息及控制模式等丰富条件，使用异步高层和动作块执行。

π_0 的基本问题是“如何从图文条件产生精细连续动作”；$\pi_{0.5}$ 进一步解决“如何使用异构监督培养可在新环境运行的高低层策略”；$\pi^*_{0.6}$ 解决“如何利用真实执行经验改进策略而不只模仿示范”；$\pi_{0.7}$ 则将过去不同任务、不同质量和不同机器人收集的行为，通过更丰富的条件组织成可以被控制的通用策略。区分离散监督、连续向量场、价值分布和最终控制动作，是贯穿这些模型的核心。